

# Normalización y SQL I

**Alex Di Genova**

**12/05/2026**

# Outline

- Normalización
- SQL

# Normalización

# Normalización

## Introducción

- Un esquema de relación por una serie de comprobaciones para certificar que satisface una determinada forma normal (Codd 1972).
- Codd propuso tres formas normales: 1FN, 2FN y 3FN
  - Basadas en las DF entre los atributos de una relación
- Posteriormente se propusieron 4FN y 5FN
  - Basadas en dependencias multivalor y concatenación.
- El proceso de normalización es un análisis de un esquema de relación, basado en sus **DF** y sus claves principales, para **garantizar** propiedades deseables como (1) **minimizar la redundancia** y (2) **minimizar las anomalías** de inserción, borrado y actualización.

# Normalización

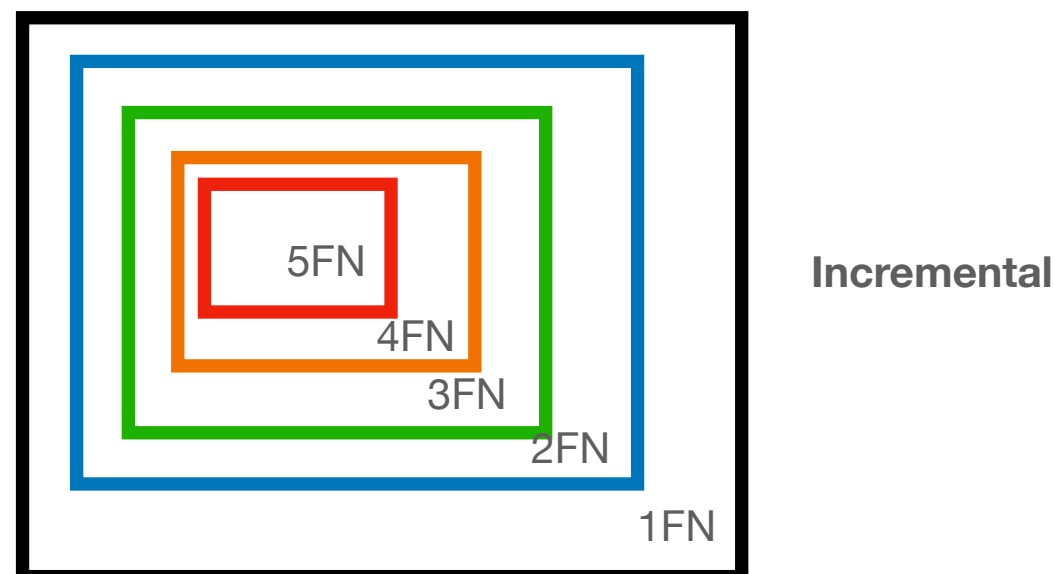
## Introducción

- Esquemas que no satisfacen las reglas de normalización, se descomponen que cumplen las reglas y por consiguiente cuentan con las propiedades deseables.
- La normalización ofrece:
  - Un marco formal para analizar esquemas de relación basado en sus claves y en las DF de sus atributos.
  - Una serie de pruebas de forma normal que pueden efectuarse sobre modelos relacionales, con el objetivo de normalizar la base de datos hasta el grado deseado (1FN, 2FN, 3FN).

# Normalización

## Propiedades

- El proceso de normalización por descomposición debe preservar la existencia de las propiedades de los esquemas relacionales.
- Propiedad de reunión no aditiva, que garantiza que no se presentará el **problema de tuplas falsas**, luego de realizar la normalización.
- Propiedad de conservación de las dependencias, que asegura que todas las **DF están representadas** en alguna relación luego de la normalización.



- En la práctica normalizamos hasta la 3FN (2FN por rendimiento)

# Normalización

## definición

- Primera forma normal (1FN)
  - Creada para prohibir atributos multivalor, compuestos y sus combinaciones.
- *Un atributo solo debe incluir valores atómicos (simples, indivisibles) y el valor de cualquier atributo en una tupla debe ser un valor simple del dominio de ese atributo.*
- Los únicos valores de atributo permitidos por la 1FN son los **atómicos** (o indivisibles)
- Soluciones:
  - Eliminar el atributo que viola la 1FN y colocarlo en una relación aparte.
  - Expandir la clave primaria ({NúmeroDepto, UbicacionesDpto}, *redundancia*)
  - Si se conoce el número máximo de valores para el atributo (N), sustituir el atributo por N atributos atómicos (valores NULL, consultas más complejas).

DEPARTAMENTO

| NombreDpto     | <u>NúmeroDpto</u> | DniDirector | UbicacionesDpto             |
|----------------|-------------------|-------------|-----------------------------|
| Investigación  | 5                 | 333445555   | {Valencia, Sevilla, Madrid} |
| Administración | 4                 | 987654321   | {Gijón}                     |
| Sede central   | 1                 | 888665555   | {Madrid}                    |



# Normalización

## Definición

- Una relación se encuentra en 2FN si:
  - Cumple la 1FN.
  - *Cada atributo no clave tiene dependencia funcional completa respecto de alguna de las claves.*
- Ejemplo:
  - $R(A,B,C,D) \text{ DF}\{A,B \rightarrow C; A \rightarrow D\}$
  - Clave candidata es  $\{A,B\} \Rightarrow$  atributos no clave C y D.
  - El atributo D depende de A pero no de una clave  $\Rightarrow$  el esquema no cumple 2FN.
  - Solución:  $R_1(A,B,C) \{A,B \rightarrow C\}$  y  $R_2(A,D)\{A \rightarrow D\}$  si cumplen 2FN.



# Normalización

## Definición

- Una relación se encuentra en 3FN si:
  - Cumple la 2FN.
  - *No existe ningún atributo no principal que dependa transitivamente de alguna clave de R*
- Ejemplo:
  - $R(A,B,C) \text{ DF}\{A \rightarrow B; B \rightarrow C\}$
  - Clave candidata es  $\{A\} \Rightarrow$  atributos no clave B y C.
  - El atributo C depende transitivamente de la clave A  $\Rightarrow$  el esquema no cumple 3FN
    - Si cumple 2FN, ya que C depende transitivamente de la clave (A).
  - Solución:  $R_1(A,B) \{A \rightarrow B\}$  y  $R_2(B,C) \{B \rightarrow C\}$  si cumplen 3FN.

# Normalización

## Resumen

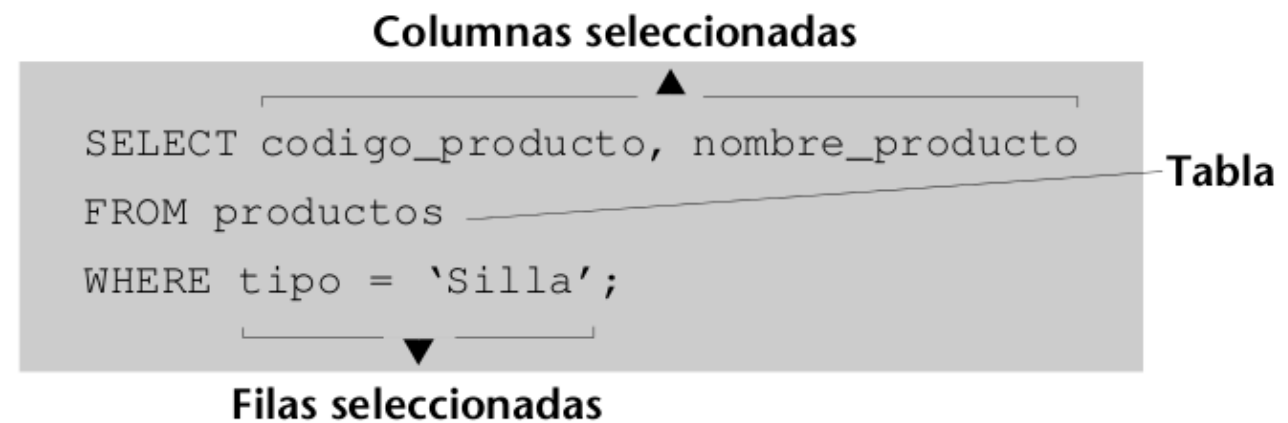
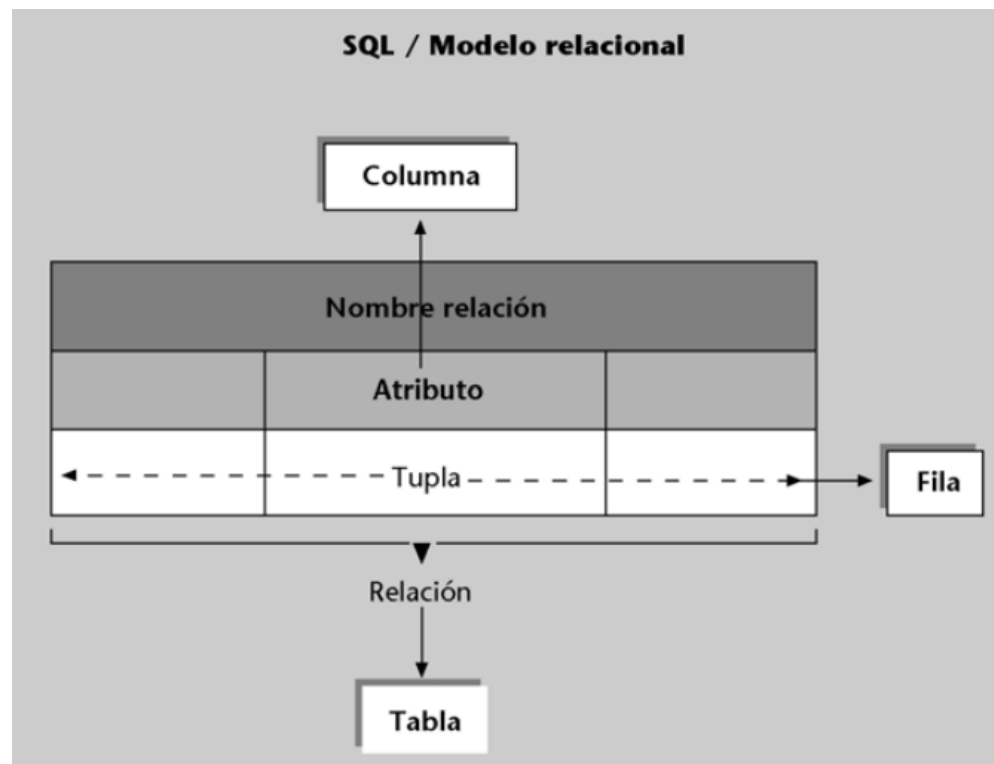
| FN  | Prueba                                                                                        | Solución                                                                                                                                     |
|-----|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| 1FN | La relación no debe tener atributos multivalor o relaciones anidadas.                         | Generar nuevas relaciones para cada atributo multivalor o relación anidada.                                                                  |
| 2FN | Cada atributo no clave tiene dependencia funcional completa respecto de alguna de las claves. | Descomponer y configurar una nueva relación por cada clave parcial con su(s) atributo(s) dependiente(s).                                     |
| 3FN | No existe ningún atributo no principal que dependa transitivamente de alguna clave de R       | Descomponer y configurar una relación que incluya el(los) atributo(s) no clave que determine(n) funcionalmente otro(s) atributo(s) no clave. |

**SQL**

# Qué es el SQL?

## Introducción

- Lenguaje estándar ANSI(1986) de **definición, manipulación y control** de bases de datos relacionales.
- Estándar de facto en la actualidad, aunque cada fabricante agrega sus propias extensiones.



# SQL

## Introducción

Nombre de la tabla

```
INSERT INTO productos  
VALUES (1250, 'LENA', 'Mesa', 'Diseño Juan Pi. Año 1920.', 25000);
```

Valores de la fila

```
CREATE TABLE productos  
(codigo_producto INTEGER,  
nombre_producto CHAR(20),  
tipo CHAR(20),  
descripcion CHAR(50),  
precio REAL,  
PRIMARY KEY (codigo_producto));
```

Nombre de la tabla

Nombre de las columnas y tipo

Clave primaria

# SQL

## Introducción

Hacer consultas

Usuario

```
GRANT SELECT ON productos TO jmontserrat;
```

Nombre de la tabla

# SQL

## Tipos de datos

- **CHARACTER** (largo) : Cadenas de caracteres de longitud fija.
- **INTEGER** : Números enteros.
- **SMALLINT** : Números enteros pequeños.
- **DECIMAL** (precisión,escala) : Número de decimales con tantos digitos como indique la precisión y tantos decimales como indique la escala.
- **DATE**: Fechas. Estan compuestas de año (YEAR), mes (MONTH) y dia (DAY)
- **BOOLEAN**: Valores verdadero o falso.

| TYPE                   | SQLite3 |
|------------------------|---------|
| INT                    | INTEGER |
| INTEGER                |         |
| TINYINT                |         |
| SMALLINT               |         |
| MEDIUMINT              |         |
| BIGINT                 |         |
| UNSIGNED BIG INT       |         |
| INT2                   |         |
| INT8                   |         |
| CHARACTER(20)          | TEXT    |
| VARCHAR(255)           |         |
| VARYING CHARACTER(255) |         |
| NCHAR(55)              |         |
| NATIVE CHARACTER(70)   |         |
| NVARCHAR(100)          |         |
| TEXT                   |         |
| CLOB                   | BLOB    |
| BLOB                   |         |
| REAL                   | REAL    |
| DOUBLE                 |         |
| DOUBLE PRECISION       |         |
| FLOAT                  |         |
| NUMERIC                | NUMERIC |
| DECIMAL(10,5)          |         |
| BOOLEAN                |         |
| DATE                   |         |
| DATETIME               |         |



# SQL

## DDL- Data Definition Language

- **DDL (lenguaje de definición de datos)**
  - Las sentencias DDL son aquellas utilizadas para la creacion de una base de datos y todos sus componentes.
  - Tablas, indices, relaciones, restricciones...
- **Sentencias**
  - CREATE -> crear tablas y vistas.
  - DROP -> para borrar tablas y vistas.
  - ALTER -> para modificar tablas.
    - ALTER TABLE EMPLEADO ADD COLUMN sueldo INT;
    - ALTER TABLE EMPLEADO DROP COLUMN mail CASCADE;
    - ALTER TABLE DEPARTAMENTO ALTER COLUMN id\_director SET DEFAULT '00000000';



# SQL

## Sentencia CREATE

```
CREATE TABLE nombre_tabla  
    ( definición_columna  
    [, definición_columna...]  
    [, restricciones_tabla]  
    );
```

Donde **definición\_columna** es:

```
nombre_columna {tipo_datos|dominio} [def_defecto] [restric_col]
```

# SQL

## Sentencia CREATE

| Restricciones de columna        |                                                                             |
|---------------------------------|-----------------------------------------------------------------------------|
| Restricción                     | Descripción                                                                 |
| NOT NULL                        | La columna no puede tener valores nulos.                                    |
| UNIQUE                          | La columna no puede tener valores repetidos. Es una clave alternativa.      |
| PRIMARY KEY                     | La columna no puede tener valores repetidos ni nulos. Es la clave primaria. |
| REFERENCES<br>tabla [(columna)] | La columna es la clave foránea de la columna de la tabla especificada.      |
| CHECK (condiciones)             | La columna debe cumplir las condiciones especificadas.                      |

**FOREIGN KEY(trackartist) REFERENCES artist(artistid)**

**FOREIGN KEY** clave\_secundaria**REFERENCES** tabla [(clave\_primaria)]  
**[ON DELETE {NO ACTION | CASCADE | SET DEFAULT | SET NULL}]**  
**[ON UPDATE {NO ACTION | CASCADE | SET DEFAULT | SET NULL}]**

| Restricciones de tabla                                                                      |                                                                                                                                                                                                                                                                         |
|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Restricción                                                                                 | Descripción                                                                                                                                                                                                                                                             |
| PRIMARY KEY<br>(columna [, columna...])                                                     | El conjunto de las columnas especificadas no puede tener valores nulos ni repetidos. Es una clave primaria.                                                                                                                                                             |
| FOREIGN KEY<br>(columna [, columna...])<br>REFERENCES tabla<br>[(columna2 [, columna2...])] | El conjunto de las columnas especificadas es una clave foránea que referencia la clave primaria formada por el conjunto de las columnas2 de la tabla dada. Si las columnas y las columnas2 se denominan exactamente igual, entonces no sería necesario poner columnas2. |
| CHECK (condiciones)                                                                         | La tabla debe cumplir las condiciones especificadas.                                                                                                                                                                                                                    |

# SQL

## DDL- Data Definition Language

The screenshot shows a web browser with multiple tabs open, including 'BED (file format)', 'Datatypes In SQLite', 'datatype sqlite3', 'SQLite Foreign Key', 'PYME\_database.ipynb', and 'uohdb/code at main'. The active tab is 'uohdb/code at main', showing the GitHub repository page for 'adigenova/uohdb'. The page header includes the GitHub logo, a search bar, and navigation links: 'Pull requests', 'Issues', 'Marketplace', and 'Explore'. The repository name 'adigenova / uohdb' is displayed with a 'Public' badge. Action buttons for 'Pin', 'Unwatch' (1), 'Fork' (2), and 'Star' (1) are visible. Below the repository name, a navigation bar shows 'Code' (selected), 'Issues', 'Pull requests', 'Actions', 'Projects', 'Wiki', 'Security', 'Insights', and 'Settings'. The main content area shows the file history for the 'main' branch. It includes a header with 'main' and 'uohdb / code /', and buttons for 'Go to file', 'Add file', and a menu icon. The file history table lists the following files and their commit details:

| File                | Commit Message                             | Commit Hash | Time Ago     |
|---------------------|--------------------------------------------|-------------|--------------|
| ..                  |                                            | d5351f5     | 12 days ago  |
| Basic_SQL.ipynb     | moving code                                |             | 2 months ago |
| Chinook_db.ipynb    | Created using Colaboratory                 |             | 2 months ago |
| ONG.ipynb           | Created using Colaboratory                 |             | 12 days ago  |
| PYME_database.ipynb | <a href="#">Created using Colaboratory</a> |             | 19 days ago  |

The footer of the page includes the GitHub logo, copyright information '© 2022 GitHub, Inc.', and a list of links: 'Terms', 'Privacy', 'Security', 'Status', 'Docs', 'Contact GitHub', 'Pricing', 'API', 'Training', 'Blog', and 'About'. The address bar at the bottom shows the URL: 'https://github.com/adigenova/uohdb/commit/8c3b5fff59328db531f2ce0ef00e621113bfa5e'.

# SQL

## DML- Data Manipulation Language

- DML (Lenguaje de manipulación de datos):
  - Utilizadas para **INSERT**, borrar **DELETE**, modificar **UPDATE** y consultar **SELECT FROM** los datos de una base de datos.

```
SELECT nombre_columnas_a_seleccionar  
FROM tabla_a_consultar  
WHERE condiciones;
```

# SQL

## SELECT

```
SELECT nombre_columnas_a_seleccionar  
FROM tabla_a_consultar  
WHERE condiciones;
```

- select \* from Jugador;
- select nombre, edad from Jugador;
- select nombre from Jugador where edad > 30;
- select nombre from Jugador where edad > 30 or id\_j < 10;

# SQL

## INSERT

```
INSERT INTO nombre_tabla [(columnas)]  
{VALUES ({v1|DEFAULT|NULL}, ..., {vn/DEFAULT/NULL})|<consulta>};
```

- insert into Jugador (id\_j, nombre, edad) values (1,"Alexis Sanchez","31");

# SQL

## DELETE

```
DELETE FROM nombre_tabla  
[WHERE condiciones];
```

- delete from jugador;
- delete from Jugador where edad > 30;



# SQL

## UPDATE

```
UPDATE nombre_tabla
SET columna = {expresión|DEFAULT|NULL}
    [, columna = {expr|DEFAULT|NULL} ...]
WHERE condiciones;
```

- update jugador set edad = 22 where name="Alexis Sanchez"



# SQL

## Funciones de agregación

- **COUNT**

- Select count(id\_j) from Jugador where edad > 31;

- **SUM**

- select sum(edad) from Jugador where edad > 31;

- **AVG**

- select avg(edad) from Jugador where edad > 31;

- **MIN y MAX**

- select max(edad) from Jugador where edad >31;

# SQL

## BETWEEN y IN

- Select \* from Jugador where edad **between** 20 and 30;
- Select \* from Jugador where id\_j **in** (1,20,35);
- Select \* from Jugador where id\_j **not in** (1,20,35);

# Practicar SQL

## Google Colab

The screenshot shows a web browser with multiple tabs open, including 'BED (file format)', 'Datatypes In SQLite', 'datatype sqlite3', 'SQLite Foreign Key', 'PYME\_database.ipynb', and 'uohdb/code at main'. The active tab is 'github.com/adigenova/uohdb/tree/main/code'. The GitHub interface shows the repository 'adigenova / uohdb' (Public) with options to Pin, Unwatch (1), Fork (2), and Star (1). The 'Code' tab is selected, showing a file list for the 'main' branch. The file list includes:

| File                | Description                                | Created      |
|---------------------|--------------------------------------------|--------------|
| ..                  |                                            |              |
| Basic_SQL.ipynb     | moving code                                | 2 months ago |
| Chinook_db.ipynb    | Created using Colaboratory                 | 2 months ago |
| ONG.ipynb           | Created using Colaboratory                 | 12 days ago  |
| PYME_database.ipynb | <a href="#">Created using Colaboratory</a> | 19 days ago  |

An arrow points to the 'Chinook\_db.ipynb' entry. The footer of the page shows the GitHub logo, copyright notice '© 2022 GitHub, Inc.', and links for Terms, Privacy, Security, Status, Docs, Contact GitHub, Pricing, API, Training, Blog, and About.

[https://github.com/adigenova/uohdb/blob/main/code/Chinook\\_db.ipynb](https://github.com/adigenova/uohdb/blob/main/code/Chinook_db.ipynb)

<https://github.com/adigenova/uohdb/commit/8c3b5fff59328db531f2ce0ef00e621113bfa5e>

# Consultas?

Consultas o comentarios?

Muchas gracias